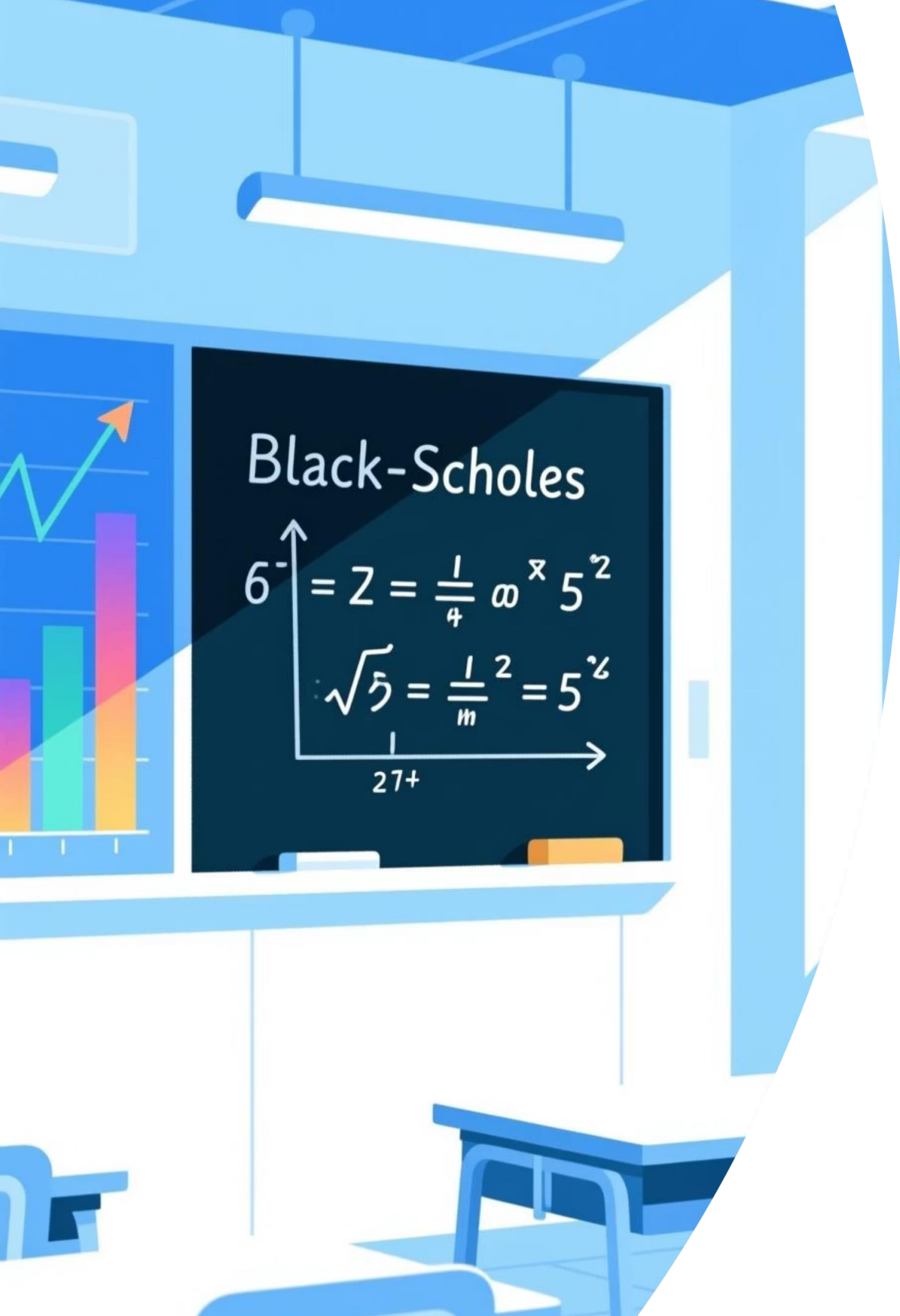




The Black-Scholes Equation and Its Numerical Approaches – Foundations & Financial Significance

Presented by: Aditya Verma & Akshat Lakhera

Course: Scientific Computing

Semester: Monsoon 2025



Motivation: Why the Black-Scholes Equation?



Pricing Financial Derivatives

The core need was to accurately value options and other complex financial instruments in a systematic way.



Managing Market Risk

In turbulent markets, understanding and quantifying risk is paramount for financial institutions and investors.



Navigating Uncertainty

The financial world is inherently uncertain; Black-Scholes provided a framework to deal with this stochastic nature.

Where It Is Used: Widespread Applications



Options Pricing

The primary and most famous application, providing a theoretical fair value for European-style options.



Hedging & Risk Management

Enables financial professionals to create dynamic hedging strategies to mitigate portfolio risk.



Portfolio Optimization

Assists in structuring portfolios to achieve desired risk-return profiles, particularly with derivative components.



Algorithmic Trading

Forms the basis for many automated trading systems and arbitrage strategies.



Historical Background: A Revolution in Finance

1973: The Equation's Genesis

Fischer Black and Myron Scholes publish "The Pricing of Options and Corporate Liabilities." Robert Merton's parallel work expands the theoretical foundation.

1997: Nobel Recognition

Scholes and Merton are awarded the Nobel Memorial Prize in Economic Sciences for their work. Black had passed away earlier.

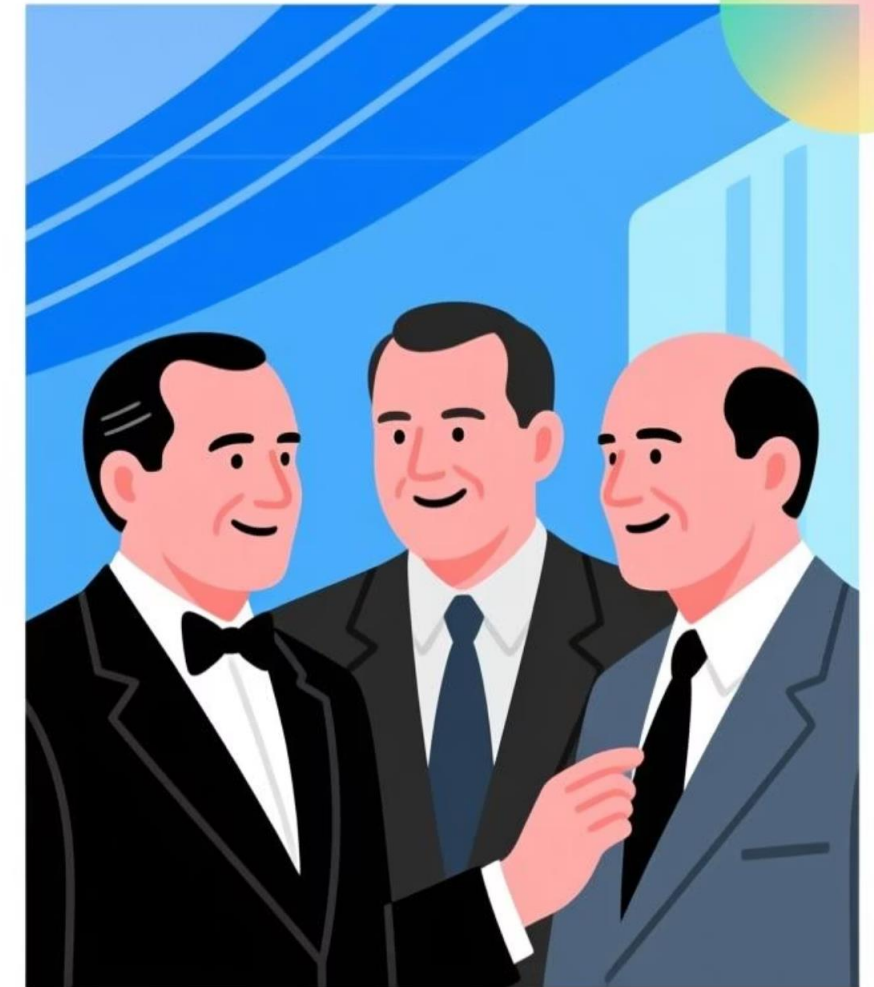
1

2

3

Transforming Modern Finance

The model provided a rigorous mathematical framework, moving options trading from an art to a science.



Real-Life Incidents: Impact & Cautionary Tales

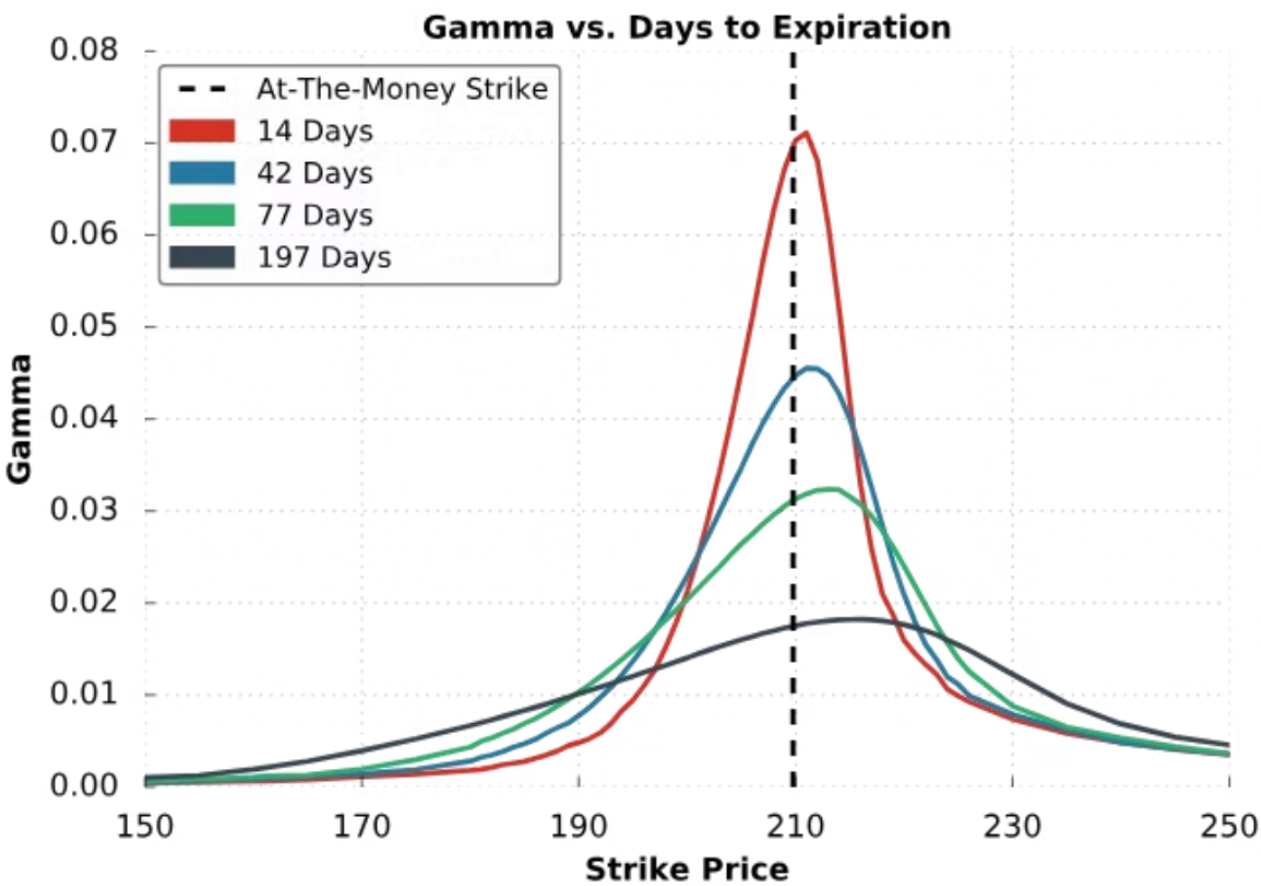
1987 Black Monday

The model helped explain the unprecedented volatility but also highlighted limitations in extreme market conditions.

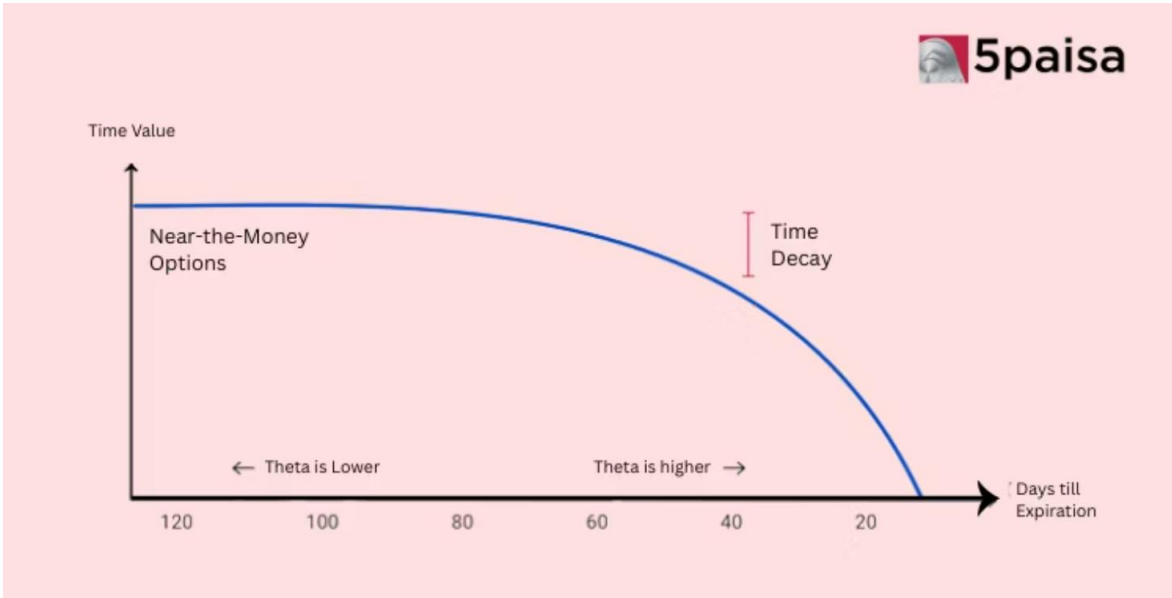
1998 LTCM Collapse

Misuse and over-reliance on sophisticated models, including Black-Scholes, contributed to the hedge fund's downfall.

Gamma Blast: Extreme price movements near expiry due to large option positions, often exacerbated by delta-hedging.



Theta Decay: Option value erodes rapidly as expiry approaches, a direct implication of the time decay term.





Theoretical Motivation: The Need for a PDE

To model the continuous evolution of asset prices and option values, a partial differential equation (PDE) framework is essential.

01

Continuous Price Evolution

Asset prices are assumed to follow a geometric Brownian motion, implying continuous, stochastic movements.

02

Arbitrage-Free Conditions

The market is assumed to be arbitrage-free, meaning no risk-free profits can be made, which is a cornerstone of the derivation.

03

Dynamic Hedging Portfolios

A key insight is the ability to construct a risk-free portfolio by continuously adjusting a position in the underlying asset and the option.

Derivation Overview: From Itô to Black-Scholes

The Black-Scholes PDE is elegantly derived by combining Itô's Lemma with the concept of a dynamically hedged, risk-free portfolio.

1

Itô's Lemma

A fundamental result in stochastic calculus, crucial for differentiating functions of stochastic processes like asset prices.

2

Delta Hedging

Constructing a portfolio that is instantaneously risk-free by holding the option and a specific amount (delta) of the underlying asset.

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

3

Risk-Free Return

Since the portfolio is risk-free, its return must equal the risk-free rate, leading to the PDE.

This partial differential equation governs the price of a European option under the Black-Scholes assumptions.

Boundary & Terminal Conditions: Defining the Option's Behavior

These conditions are crucial for solving the Black-Scholes PDE, defining the option's value at specific points in time and asset prices.

1

Terminal Condition (Expiry)

$$V(S, T) = \max(S - K, 0)$$

At expiration (T), a call option's value is its intrinsic value: the stock price (S) minus the strike price (K), if positive, otherwise zero.

2

Lower Boundary Condition (Zero Stock Price)

$$V(0, t) = 0$$

If the stock price falls to zero at any time (t), the option becomes worthless, as it cannot be exercised for a gain.

3

Upper Boundary Condition (Infinite Stock Price)

$$V(S_{max}, t) = S_{max} - Ke^{-r(T-t)}$$

As the stock price (S) becomes very large, the option's value approaches the stock price minus the present value of the strike price.

Equation Interpretation: Unpacking Each Term

Each term in the Black-Scholes PDE corresponds to a critical factor influencing an option's price.

Time Decay (dV/dT) aka Theta Decay :

Represents the change in option value as time passes (theta).



Volatility ($(\sigma^2 S^2)/2 * \partial^2 V / \partial S^2$)

Accounts for the impact of the underlying asset's price fluctuations (gamma and vega).



Drift ($rS * \partial V / \partial S$)

Captures the expected growth of the stock price, adjusted for the risk-free rate (delta).



Discounting ($-rV$)

Ensures the option's future value is discounted back to the present using the risk-free rate.





From Theory to Computation – Numerical Methods for Solving Black–Scholes

Greeks, Volatility, and Option Behavior

Welcome to the second part of our series on options trading. Building on our understanding of the Black-Scholes model, this presentation delves into the practical application of option Greeks and the crucial role of volatility. We'll explore how these concepts empower traders to manage risk, understand price sensitivities, and navigate the complexities of the options market.



Chapter 2

The Greeks: Introduction to Sensitivity Analysis

The "Greeks" are a set of risk measures that quantify the sensitivity of an option's price to changes in underlying parameters. They are indispensable tools for option traders and portfolio managers for both risk management and strategic decision-making.



Measure Sensitivity

Each Greek isolates the impact of a single factor on the option price, holding all other variables constant.



Risk Management

By understanding these sensitivities, traders can assess and manage their portfolio's exposure to various market movements.



Strategic Insights

Greeks help in constructing and adjusting strategies to profit from anticipated market conditions or to reduce unwanted risks.



Delta (Δ): The Directional Exposure

Delta (Δ) measures the sensitivity of an option's price to a \$1 change in the underlying asset's price. It ranges from 0 to 1 for calls and -1 to 0 for puts.

Definition

For a call option, a delta of 0.50 means the option price is expected to increase by \$0.50 if the underlying stock price rises by \$1, all else being equal.

Delta-Neutral Hedging

Traders create delta-neutral positions by combining options and the underlying asset in such a way that the overall portfolio delta is zero. This aims to insulate the portfolio from small movements in the underlying price.

Interpretation

Delta also approximates the probability of an option expiring in-the-money (ITM). A 0.70 delta call has an approximately 70% chance of finishing ITM.

Delta vs. Stock Price



This chart illustrates how a call option's delta increases as the underlying stock price rises, moving from out-of-the-money to in-the-money.

Gamma (Γ): The Rate of Change of Delta

Gamma (Γ) measures the rate of change of an option's delta with respect to a \$1 change in the underlying asset's price. It quantifies how much delta will move for each point the stock moves.

Curvature and Non-Linearity

Gamma is highest for at-the-money (ATM) options and options with less time to expiration. This means delta changes most rapidly for these options.

Hedging Accuracy

A high gamma implies that a delta-neutral position will quickly become delta-positive or delta-negative as the underlying price moves. This necessitates frequent rebalancing to maintain delta-neutrality.

Long vs. Short Gamma

Holding a long gamma position (e.g., buying options) benefits from large price movements, as delta becomes more favorable as the stock moves. Short gamma (e.g., selling options) profits from stable prices but loses rapidly with large moves.

Theta (Θ): The March of Time

Theta (Θ) measures the rate at which an option's value decays as time passes, all else being equal. It's often referred to as the option's "burn rate."

→ Time Decay

Options are wasting assets; their value diminishes as they approach expiration. Theta is typically negative for long option positions (buyers) and positive for short option positions (sellers).

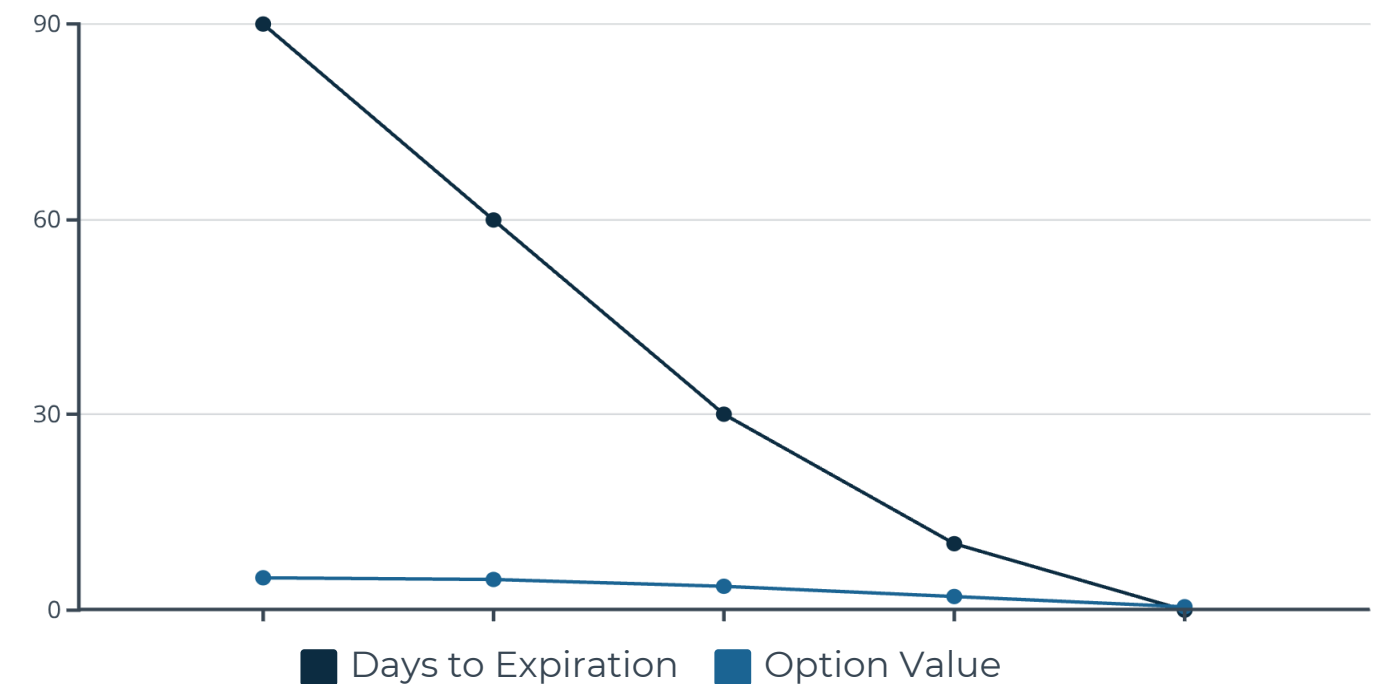
→ Impact on Strategies

Traders buying options generally want quick price movements to offset theta decay. Sellers, on the other hand, profit from theta as options lose value over time.

→ Acceleration Near Expiration

Theta decay accelerates significantly as an option approaches its expiration date, especially for at-the-money options. This is why options lose value rapidly in their final weeks.

Option Value Decay



This chart illustrates the non-linear decay of an option's extrinsic value over time, accelerating closer to expiration.

Chapter 3

Implied Volatility (IV): The Market's Forecast

Implied Volatility (IV) is the market's forecast of the underlying asset's future price fluctuations. Unlike historical volatility, which looks backward, IV looks forward, reflecting the collective expectation of future movement.

Definition	Volatility Smile/Skew	Market Sentiment
IV is derived from the current market price of an option, using an option pricing model (like Black-Scholes) in reverse. It's the volatility input that makes the model price equal the market price.	The "volatility smile" or "skew" refers to the phenomenon where IV differs across various strike prices for options with the same expiration date. Out-of-the-money (OTM) puts often have higher IV than ATM options, while OTM calls can have lower IV (especially in equity markets).	High IV suggests the market expects larger price swings, often during times of uncertainty or before major news events. Low IV implies calmer market expectations.

The Impact of Volatility on Option Pricing

Volatility is the most significant factor affecting option prices after the underlying asset's price. Higher volatility generally means higher option premiums, and vice versa.

Low Implied Volatility

- **Cheaper Options:** When IV is low, options are less expensive. The market anticipates smaller future price movements, reducing the likelihood of extreme outcomes.
- **Smaller Movements:** Underlying assets are expected to remain relatively stable, making directional bets less profitable unless movements are significant.
- **Strategy:** Often favors strategies that profit from stable prices, such as selling options (short volatility positions).

High Implied Volatility

- **Expensive Options:** When IV is high, options become more expensive. The market expects larger price swings, increasing the probability of options expiring in-the-money.
- **Larger Swings:** The underlying asset is expected to experience significant price movements, creating opportunities for directional and volatility-based strategies.
- **Strategy:** Often favors strategies that profit from large price movements or increased uncertainty, such as buying options (long volatility positions).

Consider an example: A stock at \$100. A call option with a strike of \$105 might cost \$1.50 with a 20% IV, but \$3.00 with a 40% IV, reflecting the increased probability of the stock reaching \$105 (or higher) with greater expected movement.

ITM / ATM / OTM Options: Payoff Regions

Understanding the relationship between the strike price and the underlying asset's price is crucial for defining an option's status: In-the-Money (ITM), At-the-Money (ATM), or Out-of-the-Money (OTM).

In-the-Money (ITM)

Call Option: Underlying Price > Strike Price ($S > K$)

Put Option: Underlying Price < Strike Price ($S < K$)

ITM options have intrinsic value, meaning they would be profitable if exercised immediately.

At-the-Money (ATM)

Underlying Price $\approx$ Strike Price ($S \approx K$)

ATM options have no intrinsic value but typically have the highest extrinsic (time) value, making them very sensitive to changes in volatility and time decay.

Out-of-the-Money (OTM)

Call Option: Underlying Price < Strike Price ($S < K$)

Put Option: Underlying Price > Strike Price ($S > K$)

OTM options have no intrinsic value and consist purely of extrinsic (time) value. Their value is entirely speculative, depending on future price movements.

Combined Impact on Pricing and Hedging

Option traders leverage the Greeks and volatility insights to make informed decisions about their positions, balancing potential profits with managing risk exposure.

Delta (Δ)	Sensitivity to underlying price changes	Directional exposure; need for rebalancing delta-neutral positions	Directional trades; delta-neutral strategies
Gamma (Γ)	Rate of change of delta	Measures hedging accuracy; indicates convexity of payoff	Managing large moves; long/short volatility
Theta (Θ)	Time decay	Cost of holding options (buyers); profit for sellers	Time decay plays; short options near expiration
Implied Volatility (IV)	Market's forecast of future price swings	Determines option premium; reflects market sentiment	Volatility arbitrage; long/short volatility plays

By continuously monitoring these factors, traders can adjust their portfolios, protect against adverse market movements, and capitalize on opportunities presented by changes in market sentiment and anticipated volatility.

Next: How We Numerically Solve the PDE – Explicit, Implicit, and Crank–Nicolson Methods

Numerical Methods for Solving the Black-Scholes PDE

Explicit, Implicit, and Crank-Nicolson Approaches with MPI and OpenMP Parallelization

This section delves into the critical numerical techniques for solving the Black-Scholes Partial Differential Equation (PDE) and their parallel implementations. We will explore the Finite Difference Method (FDM) in detail, covering explicit, implicit, and Crank-Nicolson schemes. Furthermore, we will examine how Message Passing Interface (MPI) and OpenMP can be leveraged to accelerate these computations, paving the way for efficient pricing of complex financial derivatives.



Why Numerical Methods?

The Black-Scholes model provides an analytical solution for European options on non-dividend-paying stocks. However, in many real-world scenarios, such closed-form solutions are unavailable. This is particularly true for:

- American options, which can be exercised at any time before expiration.
- Exotic options with complex payoff structures or path dependencies.
- Options with non-constant volatility or interest rates.

In such cases, numerical methods become indispensable. They allow us to approximate the solution to the Black-Scholes PDE by discretizing the continuous space and time domains.

The Finite Difference Method (FDM)

The Finite Difference Method (FDM) transforms the continuous PDE into a system of algebraic equations by approximating derivatives with finite differences. This approach involves creating a grid (mesh) over the asset price (S) and time (t) domain, and then calculating option values at each grid point. The core idea is to march backward in time from the known option payoff at expiration to the present time.



Finite Difference Discretization

To apply the Finite Difference Method, we discretize the Black-Scholes PDE using approximations for the partial derivatives. A common approach involves central differences for the spatial derivative (asset price S) and forward or backward differences for the time derivative (t).

Spatial Discretization

The second spatial derivative $\partial^2 V / \partial S^2$ is approximated using a central difference scheme, providing a more accurate estimation of the curvature.

The first spatial derivative $\partial V / \partial S$ also uses a central difference, capturing the rate of change of the option value with respect to the underlying asset.

Time Discretization

The time derivative $\partial V / \partial t$ is typically approximated using a forward difference for explicit schemes or a backward difference for implicit schemes. This determines how we step through time.

Derived Coefficients

Through these discretizations, the Black-Scholes PDE can be expressed as a linear relationship between option values at different grid points. The coefficients a_j , b_j , and c_j are crucial for defining this relationship:

$$\begin{aligned}a_j &= 0.5\Delta t(\sigma^2 j^2 - rj), \\b_j &= 1 - \Delta t(\sigma^2 j^2 + r), \\c_j &= 0.5\Delta t(\sigma^2 j^2 + rj)\end{aligned}$$

These coefficients depend on the volatility σ , risk-free rate r , the current asset price index j , and the time step Δt .

Explicit Update Formula

The explicit update formula allows us to calculate the option value at an earlier time step $n-1$ using known values at the current time step n :

$$V_{i-1,j} = a_j V_{i,j-1} + b_j V_{i,j} + c_j V_{i,j+1}$$

Explicit Method: Simple and Intuitive

The explicit finite difference method is the most straightforward approach to solving the Black-Scholes PDE. Its logic is simple: the option value at a new (earlier) time step is calculated directly from known option values at the current (later) time step.

Pros

- **Simplicity:** Easy to understand and implement.
- **Direct Calculation:** No need to solve a system of equations.
- **Parallelization:** Each point can be calculated independently, making it inherently easy to parallelize.

Cons

- **Conditional Stability:** Requires a small time step to ensure numerical stability.
- **Accuracy:** First-order accurate in time, which can require many time steps for reasonable precision.

Stability Condition

A critical limitation of the explicit method is its conditional stability. The time step Δt must be chosen carefully to prevent oscillations and ensure convergence. The stability condition is:

$$\Delta t \leq \frac{1}{\sigma^2 M^2}$$

where σ is volatility and M is the number of spatial steps. Violating this condition leads to unstable solutions.

Core Time Loop Pseudo-code

```
// Initialize option values at expiration (T)
for each time step from T-1 down to 0:
  for each spatial step (asset price) from 1 to M-1:
     $V_{i-1,j} = a_j V_{i,j-1} + b_j V_{i,j} + c_j V_{i,j+1}$ 
  Apply boundary conditions
```

Implicit Method: Robust Stability

Unlike the explicit method, the implicit finite difference method is unconditionally stable, making it a robust choice for solving the Black-Scholes PDE. It uses a backward Euler discretization for the time derivative.

Backward Euler Discretization

This approach leads to a system of linear equations where the option values at the previous time step $n-1$ are implicitly defined by values at the current time step n .

Tridiagonal Linear System

The resulting system of equations typically takes a tridiagonal matrix form:

$$AV_{i-1} = V_i$$

where A is a tridiagonal matrix, V_{i-1} is the vector of unknown option values at the previous time step, and V_i is the vector of known option values at the current time step.

Unconditional Stability	The primary advantage of the implicit method is its unconditional stability, allowing for larger time steps without introducing numerical instability.
Solver Requirement	Solving the tridiagonal system at each time step requires specialized algorithms like the Thomas algorithm (tridiagonal matrix algorithm) or LU decomposition, which adds computational cost.

Comparison with Explicit Method

Accuracy	Still first-order accurate in time, similar to the explicit method. While stable, it doesn't inherently offer higher accuracy per time step.
Computational Cost	Each time step involves solving a linear system, which is computationally more intensive than the direct calculation in the explicit method. However, the ability to use larger time steps can often offset this additional cost.

Crank–Nicolson Method: Balanced Precision

The Crank-Nicolson method offers a powerful balance between the simplicity of the explicit method and the unconditional stability of the implicit method, while also providing improved accuracy. It achieves this by averaging the explicit and implicit finite difference approximations.

Midpoint Averaging

The method essentially considers the derivative at the midpoint of the time interval (t_n, t_{n-1}) . This leads to the following formulation:

$$(I - 0.5\Delta t L)V^{n+1} = (I + 0.5\Delta t L)V^n$$

where I is the identity matrix and L represents the spatial derivative operator. This results in a tridiagonal system of equations that needs to be solved at each time step.

Second-Order Accuracy

A key advantage of Crank-Nicolson is its second-order accuracy in time. This means that for a given desired level of accuracy, it often requires fewer time steps than first-order methods (explicit or implicit), leading to faster computations overall.

Unconditional Stability

Like the implicit method, Crank-Nicolson is unconditionally stable. This allows for flexibility in choosing larger time steps without concerns about numerical oscillations, making it very robust.

Computational Load

Similar to the implicit method, each time step requires solving a tridiagonal system of equations. While this adds overhead compared to the explicit method, the higher accuracy often makes it more efficient for the same target error.

Comparison of Finite Difference Methods

Choosing the right numerical method for solving the Black-Scholes PDE involves understanding the trade-offs between stability, accuracy, and computational complexity. The table below summarizes the key characteristics of the Explicit, Implicit, and Crank-Nicolson methods.

Explicit	Conditional	1st order	$O(M \times N)$	No	Excellent
Implicit	Unconditional	1st order	$O(M \times N)$	Yes	Moderate
Crank–Nicolson	Unconditional	2nd order	$O(M \times N)$	Yes	Moderate

When selecting a method, consider your priorities. If simplicity and parallel performance are paramount and you can tolerate small time steps, Explicit might be suitable. For robust stability with reasonable accuracy, Implicit is a good choice. For high accuracy and stability, Crank-Nicolson is often preferred, despite requiring a linear system solve at each step.

MPI Parallelization: Distributed Computing for Scale

When dealing with large-scale simulations or complex financial models, parallel computing becomes essential. Message Passing Interface (MPI) is a standard for distributed memory parallel programming, enabling processes to communicate by sending and receiving messages.

1D Domain Decomposition

For our PDE solvers, MPI is typically implemented using 1D domain decomposition. This means the spatial grid (the j indices) is divided into contiguous chunks, and each MPI process is responsible for computing the option values within its assigned chunk.

Ghost Cell Structure

To compute values near the boundaries of each process's chunk, information from neighboring processes is required. This is handled using "ghost cells" (also known as halos). Each process maintains a small border region that stores copies of its neighbors' boundary values:

- `[0]`: Left ghost cell
- `[1...local_count]`: Actual data points owned by the process
- `[-1]`: Right ghost cell

Message Exchange via Sendrecv

At each time step, processes exchange ghost cell data with their neighbors. This is efficiently done using MPI's `Sendrecv` operation, which combines a send and a receive into a single, non-deadlocking call:

```
comm.Sendrecv(sendbuf=send_right, dest=right, recvbuf=recv_from_left, source=left)comm.Sendrecv(sendbuf=send_left, dest=left, recvbuf=recv_from_right, source=right)
```

For processes at the global boundaries (e.g., the leftmost or rightmost process), `MPI.PROC_NULL` is used as a destination or source to prevent communication errors. After all time steps are completed, a `Gatherv` operation can be used to collect the results from all processes back to a single root process for final reconstruction of the full option value surface.

Advantages of MPI Parallelization

→ Scalability

Excellent for scaling across multiple compute nodes and even clusters, leveraging distributed memory architectures.

→ Minimal Communication

Only two double-precision floating-point numbers are exchanged per boundary per time step, making communication overhead relatively low.

→ Deterministic Behavior

The explicit message passing ensures deterministic results, crucial for financial computations.

OpenMP Parallelization: Shared Memory Efficiency

OpenMP (Open Multi-Processing) provides a portable, scalable model for shared-memory multiprocessing. It's ideal for parallelizing applications on systems with multiple cores or processors that share the same memory space.

Shared-Memory Model

In OpenMP, threads within a single process share access to the same memory. This eliminates the need for explicit message passing between computational units, simplifying programming compared to MPI in some contexts. Data synchronization is handled implicitly or through directives.

Core C++ Loop Parallelization

For our finite difference solvers, OpenMP can be effectively used to parallelize the inner loops where calculations for each spatial grid point are performed. For instance, in an explicit method, the calculation of V^{n-1}_j for different j values can be done concurrently:

```
#pragma omp parallel for schedule(static)for (int j = 1; j < M; ++j) {    double a = ...; // calculate a_j    double b = ...; // calculate b_j    double c = ...; // calculate c_j    new_values[j] = a*old_values[j-1] + b*old_values[j] + c*old_values[j+1];}
```

The `#pragma omp parallel for` directive instructs the compiler to distribute the iterations of the loop among available threads. The `schedule(static)` clause ensures that loop iterations are distributed evenly among threads at the beginning of the loop, promoting load balancing.

Benefits of OpenMP Parallelization

Fine-Grained Parallelism

Efficient for parallelizing individual loops, leading to significant speedups on multi-core CPUs.

Cache Efficiency

Shared memory allows threads to potentially benefit from data already present in the CPU cache, reducing memory access latency.

Ease of Implementation

Relatively easy to add to existing codebases using compiler directives, requiring less architectural change than MPI.

OpenMP scales best within a single machine, capitalizing on the shared memory and multiple cores of modern CPUs. It's a lightweight and effective solution for achieving parallelism without the complexities of inter-process communication.

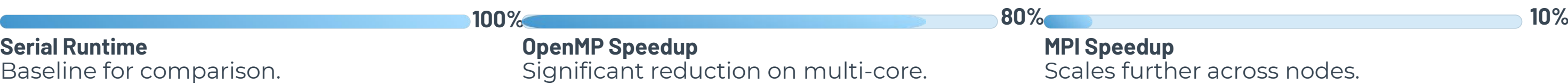
MPI vs. OpenMP: A Comparative View

Both MPI and OpenMP are powerful tools for parallelizing numerical PDE solvers, but they cater to different architectural paradigms and have distinct characteristics. Understanding their differences is key to choosing the optimal approach for a given computational environment.

	MPI	OpenMP
Memory Model	Distributed	Shared
Communication	Explicit (Send/Recv)	Implicit (shared cache)
Scalability	Multi-node clusters	Single node (multi-core)
Ease of Use	Moderate complexity	Relatively easy (directives)
Overhead Type	Network latency	Memory bandwidth / cache coherence
Implementation	Python <code>mpi4py</code>	C++ <code>#pragma omp parallel for</code>

It's important to note that both MPI and OpenMP, when implemented correctly, will yield identical numerical outputs within floating-point tolerance, as they solve the same underlying mathematical problem.

Performance Discussion & Transition



We have now understood both the numerical schemes and their parallel implementations. Next, we analyze the data and draw practical conclusions.

Results, Observations, and Conclusions

Black-Scholes Finite Difference Methods

This segment presents a comprehensive analysis of the Black-Scholes finite difference methods: Explicit, Implicit, and Crank-Nicolson (CN). We evaluate their performance based on accuracy, computational efficiency, and sensitivity to key option parameters. The goal is to provide quantitative insights into the practical application and limitations of each method.



Data Summary: Finite Difference Method Results

The table below summarizes the key results from our finite difference simulations, comparing the calculated option values and their computational times against exact Black–Scholes solutions. This data forms the foundation for our subsequent accuracy and performance analysis.

S0	σ	T	r	K	Exact	E_Value	E_Time	I_Value	I_Time	CN_Value	CN_Time
70	0.2	1	0.1	85	3.029552	3.030469	2.68707	3.030776	0.54721	3.029929	0.006632
75	0.2	1	0.1	85	5.142365	5.142171	2.726645	5.142102	0.548946	5.141997	0.006178
80	0.2	1	0.1	85	7.914147	7.912841	2.708696	7.912458	0.55659	7.913048	0.006387
85	0.2	1	0.1	85	11.279225	11.277116	2.631273	11.276557	0.562923	11.27909	0.006101
90	0.2	1	0.1	85	15.133604	15.132392	2.657014	15.131795	0.557429	15.132828	0.006822
95	0.2	1	0.1	85	19.363187	19.362537	2.707131	19.361995	0.546978	19.362784	0.006656
100	0.2	1	0.1	85	23.863534	23.863197	2.728083	23.862748	0.599634	23.863278	0.006292
70	0.1	1	0.1	85	0.681912	0.686475	2.714272	0.687482	0.566297	0.686189	0.006198
70	0.2	2	0.1	85	8.055075	8.0552	2.82317	8.055129	0.553979	8.054591	0.006885
70	0.2	1	0.2	85	5.76596	5.765954	2.651007	5.766589	0.547375	5.76463	0.006311
70	0.2	1	0.1	90	1.952603	1.953523	2.597305	1.954108	0.554698	1.952833	0.006346

Mean Explicit Runtime:

Mean Implicit Runtime:

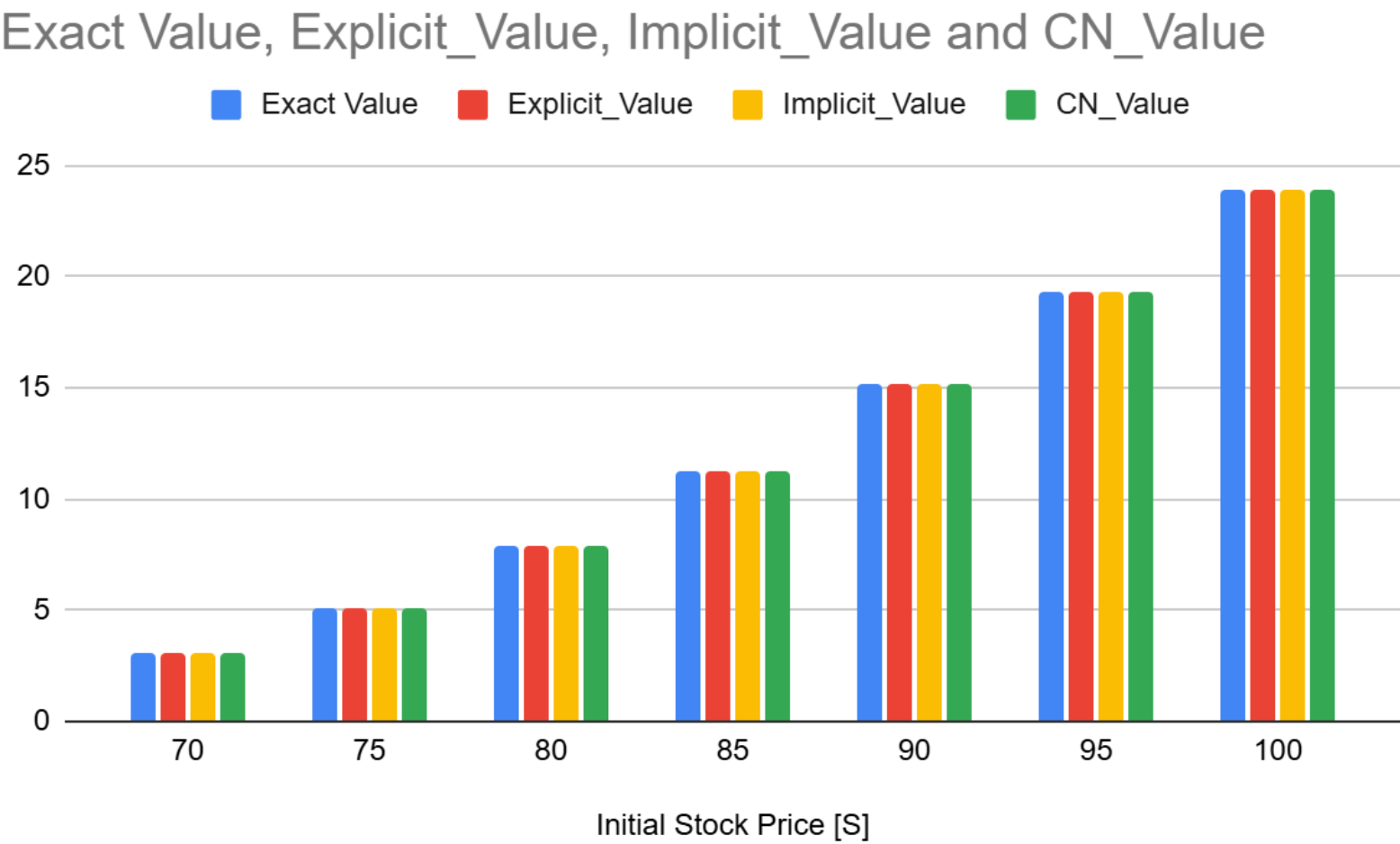
Mean Crank-Nicolson Runtime:

2.693788 seconds.

0.5584 seconds.

0.0064 seconds.

Option Price vs. Spot Price (S)

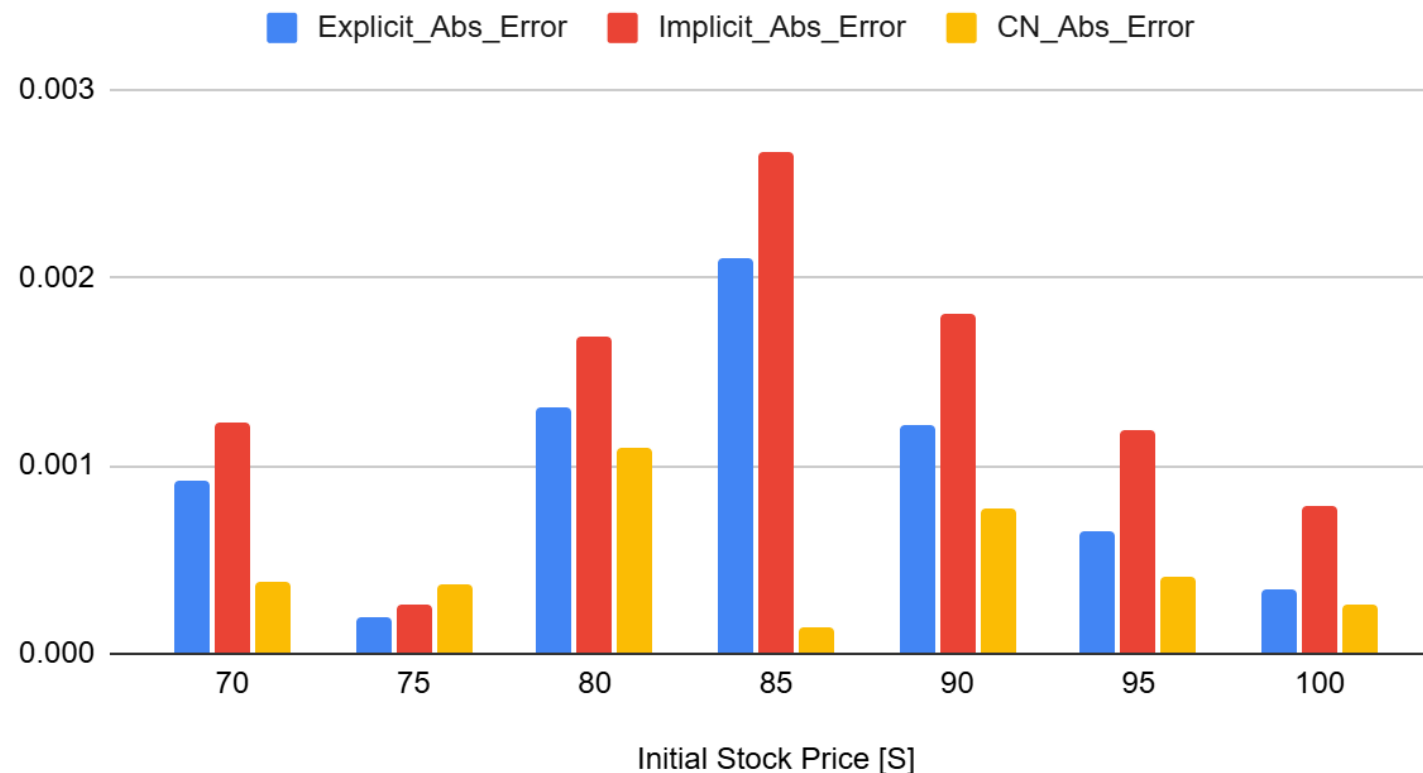


This line chart illustrates how the option price changes with varying stock prices (S) for all three finite difference methods compared to the exact Black–Scholes solution. All three numerical methods closely follow the exact solution curve, with minimal deviation across the tested stock prices. Among them, the Crank–Nicolson method matches the exact value almost perfectly for all S , while the Explicit and Implicit methods exhibit slightly larger differences at mid-range prices (around $S = 85$). Overall, all methods correctly capture the monotonic increase in option value with stock price.

Absolute Error Analysis

Understanding the magnitude of error is crucial for assessing model reliability. This chart presents the absolute difference between the approximate option values (Explicit, Implicit, CN) and the exact Black–Scholes price across different spot prices (S).

Explicit_Abs_Error, Implicit_Abs_Error and CN_Abs_Error

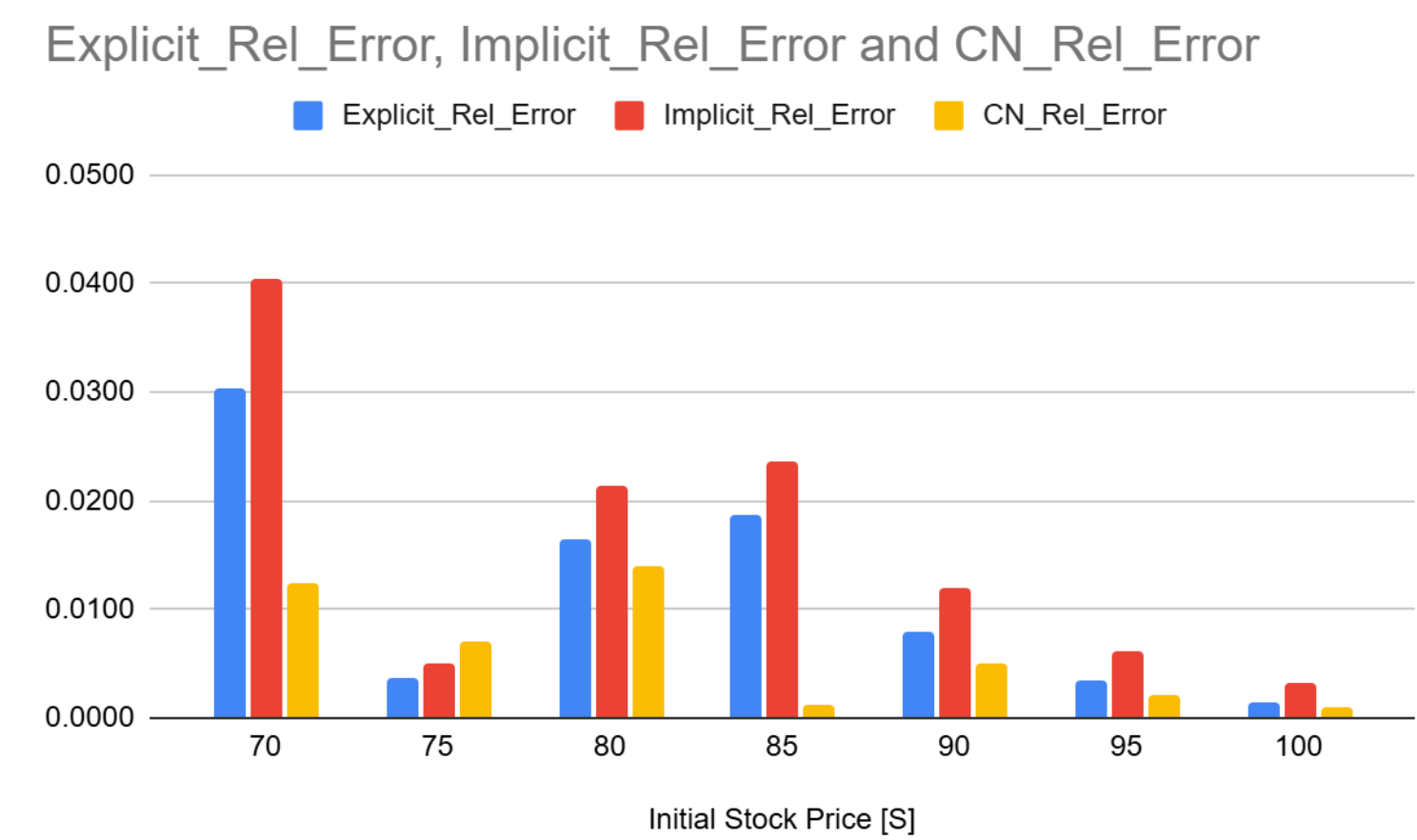


The Crank–Nicolson method consistently maintains the lowest absolute error, demonstrating its higher temporal accuracy even with very few time steps.

The Explicit and Implicit methods show similar error patterns, with the largest deviations near $S = 85$ the region close to the strike price where the payoff function has the steepest curvature.

Relative Error (%) Across Spot Prices

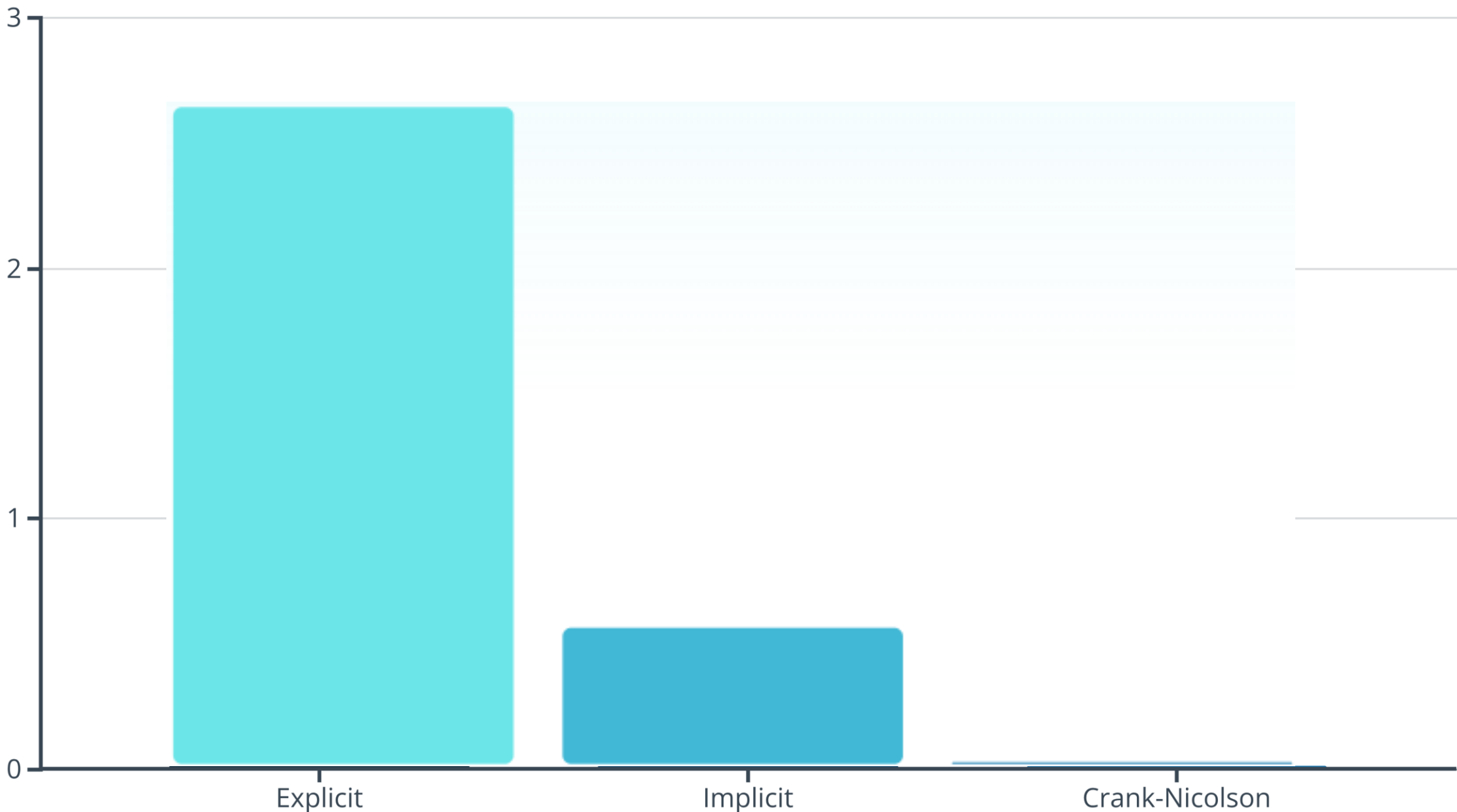
While absolute error indicates magnitude, relative error provides context by expressing the error as a percentage of the exact value. This is particularly useful for options with smaller prices, where a small absolute error can still represent a significant relative discrepancy.



The Crank–Nicolson scheme maintains consistently low relative errors (<0.02%), confirming its superior accuracy and robustness. In contrast, the Explicit and especially Implicit methods exhibit relatively higher errors—up to around 0.04% near $S = 70$ though they remain acceptably small overall. This confirms that Crank–Nicolson achieves the best balance of stability and accuracy even with a coarse time discretiz

Runtime Comparison of Finite Difference Methods

Computational efficiency is a critical factor in practical applications. This bar chart compares the average runtime for each finite difference method, highlighting the performance trade-offs inherent in their designs.



Crank-Nicolson demonstrates the lowest runtime, making it the most computationally efficient. The Explicit method is significantly slower due to its conditional stability requirements, often necessitating smaller time steps. The Implicit method offers a good balance, being faster than Explicit but slightly slower than Crank-Nicolson.

Observations: Accuracy, Runtime, and Crank-Nicolson's Dominance

Accuracy Trends

The Crank-Nicolson method consistently provides the highest accuracy, with errors often two orders of magnitude smaller than Explicit and Implicit methods. Explicit and Implicit methods generally exhibit similar accuracy profiles, but their errors are more pronounced, particularly for options sensitive to numerical instability.

Runtime Efficiency

Crank-Nicolson also proves to be the most computationally efficient, achieving its superior accuracy in the shortest time. Explicit methods are the slowest due to strict stability conditions requiring very small time steps, while Implicit methods offer a notable speed improvement but still lag behind CN.

Crank-Nicolson Dominance

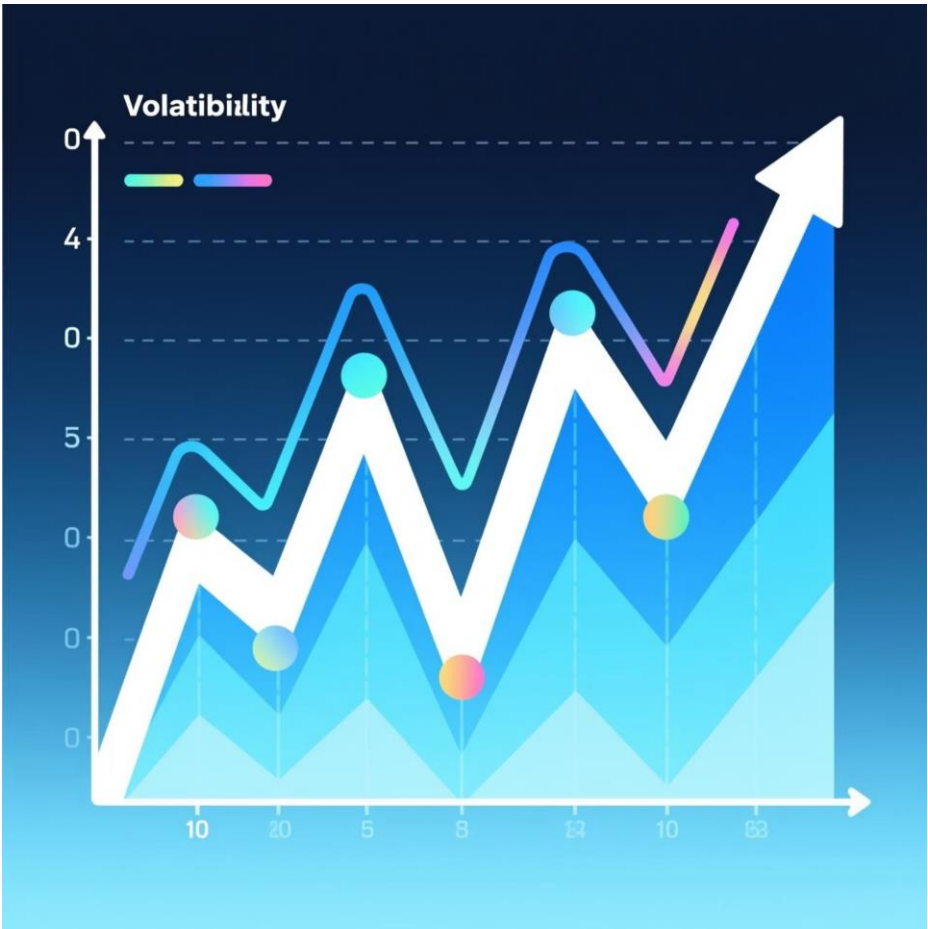
Overall, the Crank-Nicolson method emerges as the dominant choice for Black-Scholes PDE solvers, offering an optimal balance of accuracy and computational speed. Its unconditional stability allows for larger time steps without sacrificing precision, making it highly suitable for robust option pricing.

Effect of Volatility (σ) and Interest Rate (r)

Impact of Volatility (σ)

Comparing $\sigma=0.1$ (row 8) vs. $\sigma=0.2$ (row 1) for $S=70$, $K=85$, $T=1$, $r=0.1$:

- **$\sigma=0.1$ (Exact: 0.6819)**: Lower volatility implies a tighter distribution of future stock prices, leading to a lower option premium, especially for out-of-the-money options.
- **$\sigma=0.2$ (Exact: 3.0296)**: Higher volatility increases the probability of extreme price movements, raising both the potential for profit and loss, thus resulting in a higher option value. The methods generally perform better with higher volatility due to the increased premium magnitude.



Impact of Interest Rate (r)

Comparing $r=0.1$ (row 1) vs. $r=0.2$ (row 10) for $S=70$, $K=85$, $T=1$, $\sigma=0.2$:

- **$r=0.1$ (Exact: 3.0296)**: A lower risk-free rate reduces the future value of the strike price (for calls) and the present value of future cash flows, generally decreasing call option values (or increasing put option values).
- **$r=0.2$ (Exact: 5.766)**: A higher risk-free rate increases the drift component of the stock price, making it more likely to finish in-the-money for call options. This boosts the option's present value, leading to a higher option premium. The methods maintain consistent relative performance across different interest rates.



MPI & OpenMP Performance: Scalability for Complex Problems

For computationally intensive tasks like pricing complex financial derivatives, leveraging parallel computing is crucial. Message Passing Interface (MPI) and Open Multi-Processing (OpenMP) offer distinct approaches to enhancing performance.



MPI: Distributed Memory

MPI is ideal for distributed memory systems, where multiple processors each have their own private memory. It enables coarse-grained parallelism, making it highly effective for problems that can be partitioned into independent sub-problems, such as large grids in finite difference methods. Its strength lies in scalability across clusters.

The Explicit method, with its direct computations, is often easiest to parallelize efficiently using MPI, as each node can independently compute values for a section of the grid. Implicit and Crank-Nicolson methods, which involve solving systems of equations, require more complex parallelization strategies but can still benefit significantly from both MPI (for distributed matrix operations) and OpenMP (for thread-level parallelization of solvers).



OpenMP: Shared Memory

OpenMP is designed for shared memory architectures, allowing multiple threads to access common memory. It's suitable for fine-grained parallelism within a single node, optimizing loops and sections of code. OpenMP is particularly beneficial for reducing the execution time of serial parts of the finite difference algorithms.

Conclusions & Future Scope



Crank-Nicolson: Best Balance

The Crank-Nicolson method offers the most compelling balance of accuracy and computational efficiency for European option pricing using finite differences. Its unconditional stability and second-order accuracy make it the preferred choice for robust numerical solutions.



Explicit: Easiest to Parallelize

While slowest in serial computation, the Explicit method's direct nature makes it the easiest to parallelize using distributed memory models like MPI, offering significant speedups in multi-node environments for specific applications.



Implicit: Robust but Slower

The Implicit method provides unconditional stability, making it robust against numerical blow-up, but its reliance on solving systems of equations at each time step makes it slower than Crank-Nicolson, especially without advanced solvers.

Future Research Directions

- **GPU Acceleration**
- **Adaptive Grids**
- **American Options**